

Ejercicios de Práctica — Haskell Parcial

Ejercicio 1 — Pattern Matching, Guards, Tipos

Dado un sistema de notas:

```
data Calificacion = Aprobado | Desaprobado | Promocionado deriving (Show, Eq)

calificar :: Int -> Calificacion
-- <= 3: Desaprobado, 4-6: Aprobado, 7-10: Promocionado
```

Pedidos:

- a) Implementá `calificar` con **guardas**.
- b) ¿Cuál es el tipo inferido de `calificar`? ¿Y de `califica`?
- c) Implementá `esCritica :: Int -> Bool` usando **aplicación parcial** con `(<=)`.
- d) Implementá `felicitarSi :: (Int -> Bool) -> Int -> String` que devuelve "Felicitaciones" si se cumple la condición, sino "Sigue intentando".

```
data Calificacion = Aprobado | Desaprobado | Promocionado deriving (Show, Eq)

califica :: Int -> Calificacion
califica n
  | n <= 3 = Desaprobado
  | n >= 7 = Promocionado
  | otherwise = Aprobado

esCritica :: Int -> Bool
esCritica = (<= 3)

felicitarSi :: (Int -> Bool) -> Int -> String
felicitarSi condicion n
  | condicion n = "Sigue intentando"
  | otherwise = "Felicitaciones"
```

Ejercicio 2 — Data + Record Syntax + Listas

```
data Producto = Producto {
  nombreP  :: String,
  precio   :: Double,
  categoria :: String,
  stock    :: Int
```

```

} deriving (Show, Eq)

productos :: [Producto]
productos = [ Producto "Manzana" 1.5 "Fruta" 100,
              Producto "Leche" 2.0 "Lacteo" 50,
              Producto "Banana" 0.8 "Fruta" 0,
              Producto "Queso" 5.0 "Lacteo" 30 ]

```

Pedidos:

- a) `productosDisponibles :: [Producto] -> [Producto]` — filtrá los que tienen stock > 0.
- b) `productosDeLaCategoria :: String -> [Producto] -> [Producto]` — filtrá por categoría.
- c) `aplicarDescuento :: Double -> Producto -> Producto` — reducí el precio en un porcentaje (ej: 0.1 = 10%).
- d) `hayProductosCostosos :: Double -> [Producto] -> Bool` — si alguno supera el precio dado.
- e) `nombresDisponibles :: [Producto] -> [String]` — nombres de los disponibles. Resuelvelo **sin lambda**, usando composición y aplicación parcial.

```

data Producto = Producto {
    nombreP  :: String,
    precio   :: Double,
    categoria :: String,
    stock    :: Int
} deriving (Show, Eq)

productos :: [Producto]
productos = [ Producto "Manzana" 1.5 "Fruta" 100,
              Producto "Leche" 2.0 "Lacteo" 50,
              Producto "Banana" 0.8 "Fruta" 0,
              Producto "Queso" 5.0 "Lacteo" 30 ]

productosDisponibles :: [Producto] -> [Producto]
productosDisponibles = filter (\p -> stock p > 0)

productosDeLaCategoria :: String -> [Producto] -> [Producto]
productosDeLaCategoria cat = filter (\p -> categoria p == cat)

aplicarDescuento :: Double -> Producto -> Producto
aplicarDescuento desc prod = prod { precio = precio prod * (1 - desc) }

hayProductosCostosos :: Double -> [Producto] -> Bool
hayProductosCostosos limite = any (\p -> precio p > limite)

nombresDisponibles :: [Producto] -> [String]
nombresDisponibles = map nombreP . filter ((> 0) . stock)

```

Ejercicio 3 — Recursividad

Implementá las siguientes funciones **recursivamente** (sin usar las de Prelude):

- a) `miSuma :: [Int] -> Int` — suma todos los elementos.
- b) `miMaximo :: [Int] -> Int` — devuelve el máximo (podés asumir lista no vacía).
- c) `contarVeces :: Eq a => a -> [a] -> Int` — cuántas veces aparece un elemento.
- d) `intercalar :: a -> [a] -> [a]` — intercala un elemento entre cada par: `intercalar 0 [1,2,3] → [1,0,2,0,3]`.

```
miSuma :: [Int] -> Int
miSuma [] = 0
miSuma (x:xs) = x + miSuma xs

miMaximo :: [Int] -> Int
miMaximo [x] = x
miMaximo (x:xs) = max x (miMaximo xs)

contarVeces :: Eq a => a -> [a] -> Int
contarVeces _ [] = 0
contarVeces elem (x:xs)
  | elem == x = 1 + contarVeces elem xs
  | otherwise = contarVeces elem xs

intercalar :: a -> [a] -> [a]
intercalar _ [] = []
intercalar _ [x] = [x]
intercalar sep (x:xs) = x : sep : intercalar sep xs
```

Ejercicio 4 — Orden Superior + Composición (estilo examen)

```
data Superheroe = Superheroe {
  alias  :: String,
  poder  :: Int,
  equipo :: String,
  activo :: Bool
} deriving (Show)

heroes :: [Superheroe]
heroes = [ Superheroe "Iron Man" 95 "Avengers" True,
           Superheroe "Batman"  80 "JL"         True,
           Superheroe "Thor"    92 "Avengers" False,
           Superheroe "Flash"   88 "JL"         True ]
```

Pedidos:

- a) `heroesActivos :: [Superheroe] -> [Superheroe]`
- b) `poderPromedio :: [Superheroe] -> Double` — usá `sum`, `length` y `map`.
- c) `equipoMasFuerte :: String -> [Superheroe] -> Int` — suma de poder del equipo dado.

- d) `subirPoder :: Int -> Superheroe -> Superheroe` — aumenta el poder en N.
- e) `potenciarEquipo :: String -> Int -> [Superheroe] -> [Superheroe]` — subí el poder de todos los del equipo dado. Usá **composición** y **aplicación parcial**.
- f) `todosActivosDelEquipo :: String -> [Superheroe] -> Bool` — si todos los del equipo están activos.

```
heroesActivos :: [Superheroe] -> [Superheroe]
heroesActivos = filter activo

poderPromedio :: [Superheroe] -> Double
poderPromedio heroes = fromIntegral(sum (map poder heroes)) / fromIntegral(length
heroes)

equipoMasFuerte :: String -> [Superheroe] -> Double
equipoMasFuerte team = poderPromedio . filter (\h -> equipo h == team)

subirPoder :: Int -> Superheroe -> Superheroe
subirPoder pow s = s { poder = poder s + pow }

potenciarEquipo :: String -> Int -> [Superheroe] -> [Superheroe]
potenciarEquipo team pow heroes = map (subirPoder pow) . filter (\h -> equipo h ==
team) $ heroes

hallarEquipo :: String -> [Superheroe] -> [Superheroe]
hallarEquipo team = filter (\h -> equipo h == team)

todosActivosDelEquipo :: String -> [Superheroe] -> Bool
todosActivosDelEquipo team = all activo . filter (\h -> equipo h == team)
-- todosActivosDelEquipo team heroes = length(filter activo (hallarEquipo team
heroes)) == length(hallarEquipo team heroes)
```

Ejercicio 5 — Inferencia de Tipos (teórico-práctico)

¿Cuál es el tipo de cada expresión? Justificá brevemente.

```
-- a)
f x = filter (> x)

-- b)
g = map length

-- c)
h xs = foldr (+) 0 xs

-- d)
k = any even
```

```
-- e)
p f g x = f (g x)
```

Respuestas:

```
-- a) f :: Ord a => a -> [a] -> [a]
--      (> x) es (a -> Bool), filter lo convierte en [a] -> [a], con x libre queda a
--      -> [a] -> [a]

-- b) g :: [[a]] -> [Int]
--      length :: [a] -> Int, map lo aplica a una lista de listas

-- c) h :: Num a => [a] -> a
--      foldr (+) 0 acumula sumando, el 0 fija el tipo de la semilla y del resultado

-- d) k :: Integral a => [a] -> Bool
--      even :: Integral a => a -> Bool, any lo convierte en [a] -> Bool

-- e) p :: (b -> c) -> (a -> b) -> a -> c
--      Es exactamente la composicion (.), f recibe el resultado de g
```

Ejercicio 6 — foldr (avanzado)

Usá `foldr` para reimplementar:

- a) `miLength :: [a] -> Int`
- b) `miFilter :: (a -> Bool) -> [a] -> [a]`
- c) `miMap :: (a -> b) -> [a] -> [b]`
- d) `miAny :: (a -> Bool) -> [a] -> Bool`

```
miLength :: [a] -> Int
miLength = foldr (\_ acc -> acc + 1) 0

miFilter :: (a -> Bool) -> [a] -> [a]
miFilter cond = foldr (\x acc -> if cond x then x : acc else acc) []

miMap :: (a -> b) -> [a] -> [b]
miMap f = foldr (\x acc -> f x : acc) []

miAny :: (a -> Bool) -> [a] -> Bool
miAny cond = foldr (\x acc -> cond x || acc) False
```

Tips clave para el parcial

Concepto	Ejemplo rápido
Aplicación parcial	<code>filter (>5), map (*2)</code>
Composición	<code>(nombreP . head) lista</code>
Point-free	<code>f = map toUpper</code> en vez de <code>f xs = map toUpper xs</code>
Record update	<code>carta { tags = nuevos }</code>
Pattern matching en listas	<code>f (x:xs) = ...</code>
Lambda	<code>\x -> x + 1</code>

Los ejercicios 2 y 4 son los más parecidos al formato del parcial (dominio + data + varias funciones encadenadas). El 5 y 6 son los que más suelen aparecer como preguntas teórico-prácticas.